

// PYSPARK · DATABRICKS · DATA ENGINEERING

PySpark Core Concepts

`inferSchema`

`printSchema()`

Custom Schema

`dateFormat`

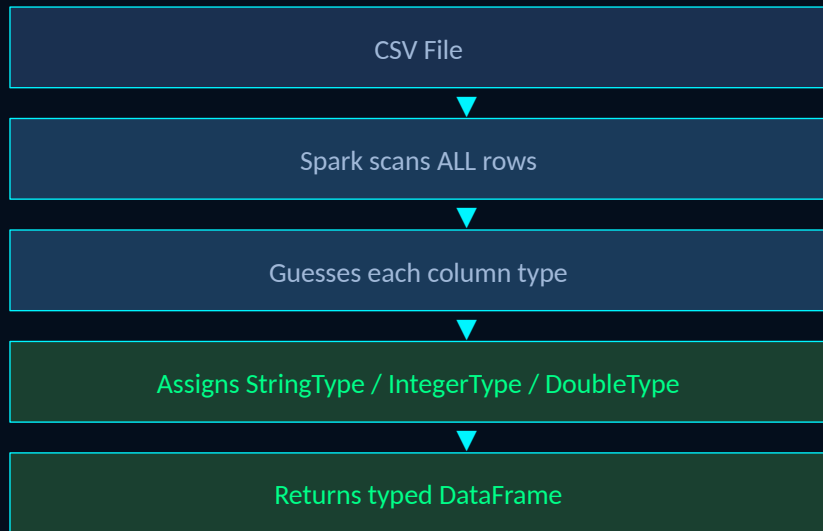
Read

Write

A concept-first guide — understand what each feature does, why it matters, and when to use it

inferSchema

What it does



```
df = spark.read \
    .option("inferSchema", True) \
    .option("header", True) \
    .csv("orders.csv")
```



Pros

Zero setup — just point at the file and Spark figures out types automatically



Cons

Full file scan = slow on large data. Dates default to StringType, not DateType!

printSchema()

What it does

Prints the schema (column names + data types + nullable flag) of a DataFrame to the console. It's a read-only inspection — it doesn't modify your data or DataFrame in any way.

```
df.printSchema()
```

```
// OUTPUT
```

```
root
|-- order_date: date      (nullable = true)
|-- country:   string    (nullable = true)
|-- order_id:  integer   (nullable = true)
|-- price:     double    (nullable = true)
```

After every read()

Always call `printSchema()` right after loading any file — verify types before doing any transformation

Debug type mismatches

If a filter or aggregation throws an error, `printSchema()` shows whether your column is string vs numeric

Before writing

Confirm final schema before writing to Parquet, Delta, or any sink — prevents schema-on-write issues

Custom Schema – StructType

Instead of letting Spark guess, you explicitly tell it: column name + data type + nullable. This is defined using StructType (the schema container) and StructField (one entry per column).

```
from pyspark.sql import types as T

csv_schema = T.StructType([
    T.StructField('order_date', T.DateType(), True),
    T.StructField('country', T.StringType(), True),
    T.StructField('order_id', T.IntegerType(), True),
    T.StructField('product', T.StringType(), True),
    T.StructField('qty', T.IntegerType(), True),
    T.StructField('price', T.DoubleType(), True),
])
```

StructType([])

The outer container — wraps ALL column definitions into a single schema object

StructField(name, type, nullable)

One definition per column. 3 args: column name, data type class, and whether nulls are allowed

T.DateType()

Tells Spark to parse this column as a proper date — critical for date math and filtering

T.IntegerType() / T.DoubleType()

Explicitly typed numbers — no guessing, no slow scan

dateFormat

When your CSV has a date column, Spark needs to know the exact format pattern to parse it correctly. The dateFormat option provides that pattern using Java SimpleDateFormat notation.

```
df = spark.read \  
  .option("header", True) \  
  .option("dateFormat", "yyyy-MM-dd") \  
  .schema(csv_schema) \  
  .csv("orders.csv")
```

Common Format Patterns

yyyy-MM-dd	2024-02-16	ISO 8601 – use this always
dd/MM/yyyy	16/02/2024	European format
MM-dd-yyyy	02-16-2024	US format
dd-MMM-yyyy	16-Feb-2024	With month abbreviation

Why it matters

Without the right pattern, Spark returns null for all date values — silently. No error, just missing data.

Works together with DateType

dateFormat only works when the column is defined as DateType in the schema. String columns ignore it.

⚠ Watch spelling!

Option key is dateFormat (capital F). A typo like dateFormar is silently ignored by Spark.

Read Operation

`spark.read` is a `DataFrameReader` — a builder object. You chain `.option()` calls to configure it, then end with a format method (`.csv()`, `.json()`, `.parquet()`) to trigger the actual read.

Supported formats

```
df = spark.read \  
  .option("header",      True)      \  
  .option("inferSchema", True)      \  
  .option("dateFormat", "yyyy-MM-dd") \  
  .schema(csv_schema)              \  
  .csv("/path/to/file.csv")
```

`.csv()`

`.json()`

`.parquet()`

`.orc()`

`.jdbc()`

`spark.read` → Entry point — returns a `DataFrameReader`

`.option("header")` → Treats row 1 as column names

`.option("inferSchema")` → Auto-detect types (use OR explicit schema, not both)

`.option("dateFormat")` → Date pattern for parsing date strings

`.schema(csv_schema)` → Attach your explicit `StructType` schema

`.csv( path )` → Triggers the read — returns a `DataFrame`

Write Operation

`df.write` is a `DataFrameWriter` — mirror of `DataFrameReader`. You configure it with `.mode()` and `.option()` calls, then trigger with a format method (`.csv()`, `.parquet()`, `.json()`, etc.).

Write Modes

```
# Write as Parquet (recommended)
df.write \
    .mode("overwrite") \
    .parquet("/path/to/output/orders")

# Write back as CSV
df.write \
    .mode("append") \
    .option("header", True) \
    .csv("/path/to/output/orders.csv")
```

"overwrite"

Delete existing data & write fresh. Most common in ETL pipelines.

"append"

Add new records to existing data. Use for incremental loads.

"ignore"

Skip write silently if destination exists. Safe for idempotent jobs.

"error"

Throw exception if path exists. DEFAULT mode — protects you from accidents.

Summary

inferSchema

- ✓ Quick exploration on small files
- ⚠ Production pipelines / large data

printSchema()

- ✓ After every `.read()` to verify types
- ⚠ Never skip it — silent type bugs are real

Custom Schema

- ✓ All production code — explicit is always better
- ⚠ Don't mix with `inferSchema`

dateFormat

- ✓ Any CSV with date columns + `DateType` in schema
- ⚠ Won't work on `StringType` columns

Read Operation

- ✓ `spark.read` → chain options → `.csv()`/`.json()`/etc.
- ⚠ Don't trust inferred types blindly

Write Operation

- ✓ `df.write.mode().format(path)` — always set mode!
- ⚠ Don't forget mode — default throws error